

MediRun

개발 규칙서

Git 규칙 · 파일 관리 · 네이밍 · 패키지 구조 · 시험 기록

TurtleBot3 Burger · ROS 2 Humble · Nav2 · Arduino · Qt(C++) · PostgreSQL

PROJECT MEMBERS

도희정 • 최치언

2026.07.23 | DEV RULES v3

1. 문서 범위와 표기

이 문서가 다루는 것

이 문서는 개발하는 동안 지켜야 할 규칙을 모은다. 앞부분(2~3장)은 Git과 파일을 다루는 작업 규칙이고, 뒷부분(4~5장)은 코드가 지켜야 할 이름과 구조 규칙이다. 일정·역할·제출 문서 매핑은 프로젝트 개요서 §4에 있고, 토픽·서비스 목록과 테이블 스키마의 실제 값은 인터페이스 명세서와 상세설계 명세서가 원본이다. 여기에는 값을 옮겨 적지 않고 기준만 둔다.

1.1 항목 표기

표기	의미
[확정]	이미 코드에 반영되어 있거나 명세서에 결정 사항으로 기록된 규칙. 바꾸려면 6.3절 절차를 따른다
[제안]	아직 코드에도 문서에도 없는 초안. 해당 주차 착수 전에 합의해 [확정]으로 올린다

1.2 공통 표기 원칙 [확정]

대상	표기	예시
패키지·노드·토픽·파라미터	snake_case	mission_manager, battery_percent
상태명·이벤트명	UPPER_SNAKE_CASE	NAVIGATING, PAYLOAD_REMOVED
C++ 클래스·Qt 위젯	PascalCase	MissionManagerNode, DashboardWindow
DB 테이블·컬럼	snake_case	missions, mission_id
브랜치명	소문자 + 하이픈	feature/aruco-docking

한글 금지: 패키지, 노드, 토픽, 파라미터, 파일명, 브랜치명, 커밋 제목에 한글을 쓰지 않는다. 한글은 주석과 문서에만 쓴다.

AURIX

2. Git 작업 규칙

2.1 브랜치 [확정]

브랜치	머지 방향	용도
main	← develop	동작 확인된 소스만. 주차 동결 시점에 태그를 붙인다
develop	← feature/* 등	통합 브랜치. 평소 작업은 여기로 모인다
feature/<기능명>	→ develop	기능 개발 (feature/aruco-docking)
fix/<내용>	→ develop	버그 수정 (fix/nav2-goal-timeout)
docs/<문서명>	→ develop	문서 작업 (docs/interface-spec)
chore/<내용>	→ develop	빌드·설정 변경 (chore/gitignore-update)

2.2 커밋 메시지 [확정]

type	설명	예시
feat	새 기능 추가	feat: add hall sensor payload monitor
fix	버그 수정	fix: keep nav2 goal while emergency stop is active
docs	문서 변경	docs: add system requirement document
test	시험 추가·기록	test(T-07): add low obstacle avoidance result
refactor	동작 변화 없는 구조 변경	refactor: separate docking manager from mission manager
chore	빌드·설정·의존성	chore: exclude rosbag from git

형식: <type>(<scope>): <subject> — scope는 선택이며 nav, vision, sensor, gui, db 중 하나를 쓴다. subject는 영문 소문자 명령형으로 쓰고 마침표를 붙이지 않는다. 시험 기록 커밋은 scope에 시험 ID를 넣는다.

2.3 PR 및 머지 [확정]

구간	셀프 머지	조건
feature/* → develop	허용	PR 생성 필수, 빌드 성공 확인, PR 본문에 작업 내용과 확인 방법 기재
develop → main	허용	PR 생성 필수, 해당 주차 완료 기준 충족, 머지 직후 동결 태그 부착

2인 팀 기준: 인원이 두 명이므로 리뷰 승인을 머지 조건으로 두지 않는다. 대신 모든 머지가 PR을 거치게 해 변경 이력이 남도록 하고, 실행 확인을 승인 대신의 조건으로 삼는다. 하나의 PR은 하나의 기능 또는 하나의 수정만 담는다.

되돌리기: 머지 후 동작이 깨지면 추가 수정보다 revert를 우선한다. main이 항상 실행 가능한 상태를 유지하는 것이 최우선이다.

2.4 주차별 동결과 태그 [확정]

시점	동결 대상	태그	이후 변경 시
1주차 말	웨이포인트 이름, 토픽 이름	v0.1-week1	6.3절 변경 절차를 따르고, 영향 받는 노드와 시험 ID를 커밋 메시지에 함께 기록한다
2주차 말	상태명, 메시지 필드	v0.2-week2	
3주차 말	GUI 화면 구성, DB 필드	v0.3-week3	
4주차 말	전체 (제출본)	v1.0	

AURIX

3. 파일 관리

3.1 커밋 대상과 제외 대상 [확정]

구분	대상	이유
커밋	지도 파일(.pgm/.yaml), 웨이포인트, 주행 파라미터	재현에 필요한 산출물이며 용량이 작다
커밋	펌웨어 .ino, 회로도, DB 스키마	하드웨어·DB 구성의 유일한 기록
커밋	시험 결과 CSV, launch · config 파일	시험 재현과 실행 재현의 근거
제외	build/ install/ log/ __pycache__/	빌드로 재생성되며 충돌을 만든다
제외	rosbag, 시연 영상, DB 덤프	용량이 크고 레포 이력을 영구히 무겁게 한다
제외	.vscode/ .idea/	개인 환경 설정

3.2 비밀정보와 환경 설정 [확정]

항목	규칙
DB 접속 정보	config/db.yaml은 커밋하지 않고 config/db.yaml.example에 키 이름만 남긴다
장비별 절대 경로	소스에 하드코딩하지 않고 launch 파라미터로 뺀다 (캘리브레이션 파일 경로, 시험 기록 파일 경로 등)
새 설정 추가 시	.example 파일을 함께 갱신하고 README 실행 절차에 반영한다

> **경로 하드코딩 금지:** 두 머신의 계정명이 달라(hj / aurix) 홈 경로를 기본값으로 박으면 다른 쪽에서 파일을 찾지 못한다. 캘리브레이션 파일이나 시험 기록 출력 파일처럼 계정 경로가 들어가는 값은 반드시 파라미터로 노출한다.

AURIX

4. 네이밍 규칙

4.1 패키지 [확정]

규칙	내용
접두	프로젝트 고유 패키지는 **medirun_**으로 시작한다
분리 기준	기능 영역 단위로 나눈다. 노드 하나마다 패키지를 만들지 않는다
구성	medirun_navigation, medirun_mission, medirun_sensor, medirun_vision, medirun_gui, medirun_db, medirun_bringup
메시지 패키지	커스텀 메시지·서비스는 medirun_msgs 하나에 모은다
시험·캘리브레이션	별도 패키지를 만들지 않고 대상 기능 영역 패키지에 넣는다. 주행 정밀도 시험은 medirun_navigation에 둔다

기존 패키지 이관: 학습 단계에서 만든 tb3_waypoint_nav, arduino_bridge, aruco_localization은 이름이 이 규칙과 다르다. 코드를 옮길 때 medirun_*로 이름을 바꾸며, 이관 시점은 1주차 말 동결 전으로 한다. 학습용 원본 패키지는 지우지 않고 그대로 둔다.

4.2 노드 [확정]

접미	역할	예시
_manager	상태를 들고 판단하는 노드	mission_manager, docking_manager, safety_manager, waypoint_manager
_bridge	ROS 2 밖의 것과 연결하는 노드	sos_bridge, serial_bridge
_node	단일 기능을 수행하는 노드	aruco_detector_node, led_status_node, voice_notifier_node, rotation_test_node
접미 없음	고유 역할이 이름에 이미 드러나는 경우	emergency_dispatcher, event_logger

4.3 토픽·서비스·액션 [확정]

구분	형식	예시
프로젝트 토픽	/medirun/<도메인>/<대상>	/medirun/mission/state, /medirun/sensor/payload
도메인 목록	mission · sensor · vision · safety · ui · hw	

구분	형식	예시
서비스	동사로 끝낸다	/medirun/mission/cancel, /medirun/safety/reset
액션	동작 명사	/medirun/vision/dock
표준 토픽	바꾸지 않는다	/cmd_vel, /scan, /odom, /amcl_pose, /tf

외부 패키지 토픽: Nav2, TurtleBot3, 카메라 패키지가 이미 정한 토픽 이름은 그대로 쓴다. 리매핑으로 /medirun/ 밑에 억지로 넣지 않는다. 규칙은 우리가 새로 만드는 토픽에만 적용한다.

4.4 파라미터·프레임 [확정]

대상	규칙	예시
파라미터 키	snake_case, 단위를 이름에 포함	obstacle_stop_distance_m, payload_debounce_sec
임계값 파라미터	하드코딩 금지, launch 또는 yaml로 노출	battery_reject_percent
TF 프레임	REP-105 표준 유지	map, odom, base_link, base_scan
추가 프레임	부착 위치를 이름에	camera_link, aruco_marker

4.5 상태명·식별자 [확정]

대상	형식	예시
미션 상태	대문자, 한글로 번역하지 않음	IDLE, VALIDATING, DISPATCHED, NAVIGATING, ARRIVED, PAYLOAD_REMOVED, RETURNING_HOME, DOCKED, COMPLETED, FAILED
미션 식별자	UUID	mission_id
버튼 식별자	BTN_ + 두 자리	BTN_01 ~ BTN_03
웨이포인트 식별자	DEMO_WP_ + 두 자리, 홈은 HOME	DEMO_WP_02, HOME
DB enum	DB에도 대문자 그대로 저장하며 변환하지 않는다	status = 'COMPLETED'

5. 패키지 및 레포 구조

단일 레포 구성

VM(hj)과 Pi(aurix) 두 머신을 쓰지만 레포는 하나로 가져간다. 머신마다 레포를 나누면 같은 인터페이스를 두 곳에서 고치게 되고 어느 쪽이 최신인지 알 수 없게 된다. 머신별 차이는 빌드 범위와 실행 대상으로만 구분한다. 최상위 폴더는 아래 일곱 개로 고정하며, 새로운 성격의 산출물이 생기면 최상위를 늘리지 않고 해당 폴더의 하위에 한 겹을 둔다.

5.1 레포 최상위 구조 [확정]

경로	내용
src/	ROS 2 패키지 전체 (medirun_*, medirun_msgs). 빌드 대상이 아닌 것은 두지 않는다
firmware/	Arduino 펌웨어. 스케치명과 같은 이름의 폴더를 한 겹 둔다
maps/	지도 .pgm/.yaml, 웨이포인트 정의
config/	DB·장비 설정. 실값 파일은 커밋하지 않고 .example만 둔다
db/	PostgreSQL 스키마, 마이그레이션
docs/	제출 문서, 시험표, 회로도, 시험 측정 원본 (5.2절)
scripts/	자동 실행 설정, 실행 스크립트, 사후 분석 도구 (5.2절)

5.2 보조 폴더 하위 구조 [확정]

경로	내용	분리 이유
scripts/	기동·자동 실행 스크립트. 로봇을 움직이는 시점에 실행한다	실행 시점이 다르다. 기동 스크립트는 현장에서, 분석 도구는 측정이 끝난 뒤 책상에서 돌린다
scripts/analysis/	측정값 통계·그래프 등 사후 분석 도구. ROS에 의존하지 않는다	
docs/tests/	시험 기록 문서. 조건·결과·판정을 사람이 읽는 형태로 남긴다	갱신 주기가 다르다. 측정 원본은 시행마다 늘어나고 문서는 시험이 끝난 뒤 한 번 쓴다
docs/tests/data/	시험 측정 원본 CSV. 가공하지 않은 값을 그대로 둔다	

최상위 신설 금지: 분석 도구를 tools/에, 측정 원본을 data/에 두는 방식은 쓰지 않는다. 최상위가 늘어나면 5.1절 표가 실제 레포와 어긋나고, 이후에 생기는 산출물마다 같은 판단을 반복하게 된다.

5.3 패키지 내부 구조 [확정]

경로	내용
<패키지명>/	노드 소스 (Python) 또는 src/ (C++)
launch/	런치 파일. 패키지 루트가 아니라 반드시 이 폴더에 둔다
config/	노드 파라미터 yaml
package.xml · setup.py	패키지 선언과 의존성

패키지 안에 두지 않는 것: 분석 스크립트와 측정 원본은 패키지 안에 두지 않는다. colcon이 빌드마다 install/로 복사해 산출물이 이중으로 남고, 데이터 이력이 코드 이력에 섞인다. 5.2절 위치를 쓴다.

5.4 머신별 실행 구분 [확정]

패키지	실행 머신	비고
medirun_navigation	VM	Nav2 의존
medirun_mission	VM	Nav2 액션 사용
medirun_vision	VM	Pi 카메라 토픽을 구독해 연산
medirun_sensor	Pi	아두이노 시리얼 직결. nav2_msgs 없음
medirun_gui	VM	Qt 대시보드
medirun_db	VM	PostgreSQL 접속
medirun_bringup	양쪽	런치 인자로 머신을 구분한다

AURIX

6. 기록 규칙

6.1 시험 ID 체계 [확정]

ID	시험 항목	대응 시나리오	기록
T-01	웨이포인트 반복 도달 정밀도	S-01	시험표에 일시, 조건, 결과, 판정을 남기고 커밋 scope에 ID를 적는다
T-05	버튼 ↔ 웨이포인트 매핑 3회 반복	S-01, S-07	
T-07	하단 장애물 감속·정지·재계획	S-08	
T-12	ArUco 도킹 정밀도 및 재시도	S-09	

ID 부여 규칙: 시험 ID는 **T- + 두 자리** 일련번호로 붙이고 한 번 부여한 번호는 재사용하지 않는다. 시나리오서의 S-xx는 참조 관계로만 적으며, 하나의 시나리오에 여러 시험이 붙을 수 있다. 위 표는 예시이며 전체 목록은 시험표가 원본이다.

시험 산출물 파일명: 시험 문서와 측정 원본은 파일명을 시험 ID로 시작한다. 예를 들어 T-20 시험은 docs/tests/T-20_rotation_accuracy.md와 docs/tests/data/T-20_rotation_log.csv가 된다. 문서·데이터·커밋 scope가 같은 ID로 묶여야 수치의 출처를 되짚을 수 있다.

6.2 README 규칙 [확정]

항목	내용
위치	레포 루트에 하나만 둔다. 패키지별 README는 만들지 않는다
필수 내용	프로젝트 소개, 서비스 시나리오, 하드웨어·소프트웨어 구성, 패키지 구조, 설치·빌드·실행 방법, 자동 실행 방법, 토픽·서비스·액션 목록, 지도와 웨이포인트 설명, 시험 결과, 알려진 문제, 팀원 역할
갱신 시점	주차 동결 시점마다 갱신한다. 실행 절차가 바뀌는 변경은 같은 PR에서 README도 함께 고친다

6.3 규칙 변경 절차

01 제안
변경 사유 정리
무엇을 어떤 이름으로 바꾸는지, 왜 지금 바꿔야 하는지 한 줄로 적는다. 취향 차이는 사유가 되지 않는다.

02 확인
영향 범위 확인
해당 이름을 쓰는 노드, launch 파일, GUI, DB 필드를 모두 찾는다. 하나라도 빠지면 실행 시점에 깨진다.

03 반영
코드·문서 동시 수정

04 기록
변경 기록

코드와 인터페이스 명세서, 이 문서를 같은 PR에서 함께 고친다. 문서를 나중에 고치면 반드시 어긋난다.

커밋 메시지에 영향 받는 노드와 관련 시험 ID를 적고, 아래 변경 이력 표에 한 줄 추가한다.

6.4 문서 변경 이력

날짜	변경 대상	내용
2026.07.23	5장 · 6.1 · 4.1	v3. 보조 폴더 하위 구조를 5.2절로 신설(scripts/analysis/, docs/tests/data/)하고 기존 5.2·5.3을 5.3·5.4로 밀었다. 4.1에 시험·캘리브레이션 코드의 소속 규칙을, 6.1에 시험 산출물 파일명 규칙을 추가했다. 최상위 폴더 구성은 변경하지 않았다
2026.07.22	전체	개편. 일정·역할·제출 문서 매핑을 프로젝트 개요서 §4로 이관하고, 코드 컨벤션(4~5장)을 이 문서로 통합. 문서명을 개발 규칙서로 변경
2026.07.20	전체	초판 (개발·형상관리 규칙서)

[제안] 항목 처리: 현재 이 문서에 [제안] 항목은 없다. 새 규칙을 제안할 때는 항목 옆에 [제안]과 확정 목표 시점을 함께 적고, 합의 후 [확정]으로 바꾸며 위 이력에 한 줄 남긴다.

AURIX